Une base relationnelle contient des **tables**.
Chaque table ressemble à un tableau :

- les **colonnes** décrivent les types d'informations ;
- les **lignes** correspondent aux enregistrements.

Exemple :

Table eleves

id_eleve	nom	prenom
1	Martin	Léa
2	Durand	Hugo
3	Petit	Inès

Table classes

id_classe	nom_classe	professeur
10	TG1	Mme Dupont
11	TG2	M. Leroy

Table inscriptions

id_eleve	id_classe
1	10
2	11
3	10

Ici, les données sont séparées en plusieurs tables.
C'est justement pour ça que JOIN existe : il permet de **relier** ces tables entre elles.

Pourquoi a-t-on besoin de JOIN ?

Quand on débute, on imagine souvent qu'une seule table pourrait tout contenir. Par exemple :

id_eleve	nom	prenom	nom_classe	professeur
1	Martin	Léa	TG1	Mme Dupont
2	Durand	Hugo	TG2	M. Leroy
3	Petit	Inès	TG1	Mme Dupont

Ça semble simple… mais ce n'est pas une bonne idée.

Pourquoi ?

Parce qu'on répète plusieurs fois les mêmes informations :

- la classe TG1 apparaît plusieurs fois ;
- le nom du professeur aussi ;
- si on change le professeur de TG1, il faut modifier plusieurs lignes.

En base relationnelle, on préfère donc :

- séparer les informations en plusieurs tables ;
- relier ces tables avec des identifiants.

C'est plus propre, plus fiable, plus logique.

Mais du coup, quand on veut retrouver des informations complètes, il faut **recomposer les liens** entre les tables.
Et c'est exactement le rôle de JOIN.

L'idée intuitive de JOIN

JOIN signifie en gros : **joindre**, **raccrocher**, **mettre en relation**.

On peut le voir comme ceci :

- dans une table, on a une information partielle ;
- dans une autre table, on a une autre information partielle ;
- grâce à une colonne commune, on peut réunir les deux.

Autrement dit :

JOIN sert à associer des lignes de deux tables quand elles correspondent selon une condition.

Cette condition est écrite avec ON.

Structure générale :

```
SELECT ...
FROM table1
JOIN table2
ON condition_de_correspondance;
```

Le point essentiel : la condition ON

C'est ici que les élèves se perdent souvent.

Dans un JOIN, la condition ON indique :

"quelles lignes de la première table vont avec quelles lignes de la deuxième table"

Exemple simple :

```
SELECT *
FROM inscriptions
JOIN eleves
ON inscriptions.id_eleve = eleves.id_eleve;
```

Ici, on dit :

- on prend la table inscriptions,
- on la joint avec la table eleves,
- et on associe les lignes quand id_eleve est le même dans les deux tables.

Donc :

- la ligne id_eleve = 1 de inscriptions sera associée à la ligne id_eleve = 1 de eleves,
- idem pour 2,
- idem pour 3.

Visualiser ce que fait le JOIN

Prenons les deux tables :

inscriptions

id_eleve	id_classe
1	10
2	11
3	10

eleves

id_eleve	nom	prenom
1	Martin	Léa
2	Durand	Hugo
3	Petit	Inès

Requête :

```
SELECT *
FROM inscriptions
JOIN eleves
ON inscriptions.id_eleve = eleves.id_eleve;
```

Résultat :

id_eleve	id_classe	id_eleve	nom	prenom
1	10	1	Martin	Léa
2	11	2	Durand	Hugo
3	10	3	Petit	Inès

Le JOIN a fabriqué une nouvelle table résultat, en rassemblant les colonnes des deux tables.

Ce que signifie vraiment la condition de jointure

Il ne faut pas lire :

ON inscriptions.id_eleve = eleves.id_eleve

comme une simple égalité mathématique isolée.

Il faut la lire comme :

“On associe une ligne de inscriptions avec une ligne de eleves lorsqu’elles ont le même id_eleve.”

C’est une **règle d’appariement**.

C’est le cœur de l’idée.

Une analogie simple

Imagine deux listes papier :

Liste A : les copies rendues

id_eleve	devoir
1	DM1
2	DM1

Liste B : les élèves

id_eleve	nom
1	Martin
2	Durand
3	Petit

Si tu veux savoir **qui** a rendu le devoir, tu relies les deux listes par id_eleve.

C’est exactement un JOIN.

Pourquoi ne pas écrire juste WHERE ?

Très bonne question, et source fréquente de confusion.

On peut parfois voir :

```
SELECT *
FROM inscriptions, eleves
WHERE inscriptions.id_eleve = eleves.id_eleve;
```

Cela marche aussi dans beaucoup de systèmes SQL.
C'est une ancienne manière d'écrire une jointure.

Mais aujourd'hui, on préfère écrire :

```
SELECT *
FROM inscriptions
JOIN eleves
ON inscriptions.id_eleve = eleves.id_eleve;
```

Pourquoi ?

Parce que c'est :

- plus lisible ;
- plus clair ;
- plus logique ;
- plus facile à enseigner.

Avec JOIN ... ON, on distingue bien :

- la partie qui sert à **relier les tables** ;
- et la partie WHERE, qui sert à **filtrer le résultat**.

Bien distinguer ON et WHERE

C'est un point très important.

ON

sert à dire :
comment on relie les tables

WHERE

sert à dire :
quelles lignes du résultat final on garde

Exemple :

```
SELECT eleves.nom, classes.nom_classe
FROM inscriptions
JOIN eleves
```

```
ON inscriptions.id_eleve = eleves.id_eleve
JOIN classes
ON inscriptions.id_classe = classes.id_classe
WHERE classes.nom_classe = 'TG1';
```

Ici :

- ON inscriptions.id_eleve = eleves.id_eleve
  relie inscriptions et eleves
- ON inscriptions.id_classe = classes.id_classe
  relie inscriptions et classes
- WHERE classes.nom_classe = 'TG1'
  ne garde que les élèves de TG1

Donc :

- ON relie,
- WHERE filtre.

Exemple complet pas à pas

On a trois tables :

eleves

id_eleve	nom	prenom
1	Martin	Léa
2	Durand	Hugo
3	Petit	Inès

classes

id_classe	nom_classe
10	TG1
11	TG2

inscriptions

id_eleve	id_classe
1	10
2	11
3	10

On veut obtenir :

- le nom de l'élève
- et le nom de sa classe

Requête :

```
SELECT eleves.nom, eleves.prenom, classes.nom_classe
FROM inscriptions
JOIN eleves
ON inscriptions.id_eleve = eleves.id_eleve
JOIN classes
ON inscriptions.id_classe = classes.id_classe;
```

Résultat :

nom	prenom	nom_classe
Martin	Léa	TG1
Durand	Hugo	TG2
Petit	Inès	TG1

Pourquoi passe-t-on souvent par une table intermédiaire ?

Parce qu'il n'y a pas toujours un lien direct entre deux tables.

Ici :

- eleves contient les élèves ;
- classes contient les classes ;
- inscriptions indique qui est dans quelle classe.

La table inscriptions joue le rôle de **pont**.

C'est très courant dans les bases de données.

On parle souvent de :

- relation directe ;
- ou table d'association.

Clé primaire et clé étrangère

Pour comprendre les JOIN, il faut aussi bien sentir deux notions.

Clé primaire

C'est l'identifiant unique d'une table.

Exemples :

- eleves.id_eleve
- classes.id_classe

Chaque ligne a une valeur unique pour cette colonne.

Clé étrangère

C'est une colonne qui contient l'identifiant d'une ligne d'une autre table.

Dans inscriptions :

- id_eleve renvoie vers eleves.id_eleve
- id_classe renvoie vers classes.id_classe

C'est justement grâce à ces clés étrangères qu'on peut faire des jointures.

Le INNER JOIN : le plus courant

Quand on écrit simplement :

JOIN

on veut généralement dire :

INNER JOIN

C'est la jointure interne.

Elle garde uniquement les lignes qui ont une correspondance dans les deux tables.

Exemple :

```
SELECT *
FROM inscriptions
INNER JOIN eleves
ON inscriptions.id_eleve = eleves.id_eleve;
```

S'il y a une inscription avec un id_eleve qui n'existe pas dans eleves, cette ligne ne sera pas gardée.

Exemple avec une ligne sans correspondance

inscriptions

id_eleve	id_classe
1	10
2	11
99	10

eleves

id_eleve	nom
1	Martin
2	Durand

Requête :

```
SELECT *
FROM inscriptions
JOIN eleves
ON inscriptions.id_eleve = eleves.id_eleve;
```

Résultat :

id_eleve	id_classe	id_eleve	nom
1	10	1	Martin
2	11	2	Durand

La ligne avec 99 disparaît, car il n'y a pas de correspondance dans eleves.

Et le LEFT JOIN alors ?

Le LEFT JOIN garde **toutes les lignes de la table de gauche**, même s'il n'y a pas de correspondance à droite.

Exemple :

```
SELECT *
FROM inscriptions
LEFT JOIN eleves
ON inscriptions.id_eleve = eleves.id_eleve;
```

Résultat :

id_eleve	id_classe	id_eleve	nom
1	10	1	Martin
2	11	2	Durand
99	10	NULL	NULL

Cette fois, la ligne 99 reste, mais les colonnes venant de eleves valent NULL.

Pour des élèves fragiles, je conseille souvent de bien installer d'abord :

- JOIN = INNER JOIN
- puis seulement ensuite LEFT JOIN

L’erreur classique : oublier la condition de jointure

Quand on met deux tables ensemble sans vraie condition de correspondance, on peut obtenir un **produit cartésien**.

Exemple dangereux :

```
SELECT *
FROM eleves
JOIN classes;
```

ou ancienne écriture :

```
SELECT *
FROM eleves, classes;
```

Sans condition correcte, chaque élève est associé à chaque classe.
On obtient toutes les combinaisons possibles.

Si on a :

- 3 élèves
- 2 classes

on obtient 6 lignes.

C’est presque toujours une erreur quand on débute.

L’autre erreur classique : joindre sur les mauvaises colonnes

Exemple faux :

```
SELECT *
FROM inscriptions
JOIN classes
ON inscriptions.id_eleve = classes.id_classe;
```

Ici, on compare :

- un identifiant d’élève
- avec un identifiant de classe

Ça n’a pas de sens.

Quand on fait un JOIN, il faut toujours se demander :

“Quelle colonne de cette table correspond logiquement à quelle colonne de l’autre table ?”

Autrement dit :

- élève avec élève ;
- classe avec classe ;
- commande avec client ;

- livre avec auteur, selon la structure choisie.

Pourquoi utiliser des noms de table devant les colonnes ?

Dans les jointures, on écrit souvent :

inscriptions.id_eleve
eleves.id_eleve

plutôt que simplement :

id_eleve

Pourquoi ?

Parce que plusieurs tables peuvent avoir une colonne du même nom.

Par exemple :

- inscriptions.id_eleve
- eleves.id_eleve

Si on écrivait seulement id_eleve, SQL ne saurait pas toujours laquelle choisir.

C'est donc plus clair et plus sûr de préciser le nom de la table.

Les alias : pour alléger l'écriture

Quand les requêtes deviennent plus longues, on peut utiliser des alias.

Exemple :

```
SELECT e.nom, e.prenom, c.nom_classe
FROM inscriptions i
JOIN eleves e
ON i.id_eleve = e.id_eleve
JOIN classes c
ON i.id_classe = c.id_classe;
```

Ici :

- i représente inscriptions
- e représente eleves
- c représente classes

C'est exactement la même logique, mais plus court.

Pour des élèves, c'est utile plus tard, quand la structure est déjà bien comprise.

Une méthode simple pour construire une requête avec JOIN

Quand un élève bloque, je conseille cette méthode en 4 questions :

1. Quelles informations veux-tu afficher ?

Exemple :

- le nom de l'élève
- le nom de la classe

Donc le SELECT portera sur :

SELECT eleves.nom, classes.nom_classe

2. Dans quelles tables se trouvent ces informations ?

- nom est dans eleves
- nom_classe est dans classes

3. Comment ces tables sont-elles reliées ?

Souvent, pas directement.
Il faut passer par inscriptions.

4. Quelles colonnes permettent la correspondance ?

- inscriptions.id_eleve = eleves.id_eleve
- inscriptions.id_classe = classes.id_classe

Et on écrit la requête.

Exemple guidé complet

Question : **Afficher le prénom des élèves et leur classe.**

Étape 1 : informations demandées

- prénom → table eleves
- classe → table classes

Étape 2 : chemin entre les tables

eleves ↔ inscriptions ↔ classes

Étape 3 : conditions de jointure

- inscriptions.id_eleve = eleves.id_eleve
- inscriptions.id_classe = classes.id_classe

Requête

SELECT eleves.prenom, classes.nom_classe
FROM inscriptions
JOIN eleves

```
ON inscriptions.id_eleve = eleves.id_eleve
JOIN classes
ON inscriptions.id_classe = classes.id_classe;
```

Exemple avec filtre

Question : **Afficher les noms des élèves de la classe TG1.**

```
SELECT eleves.nom
FROM inscriptions
JOIN eleves
ON inscriptions.id_eleve = eleves.id_eleve
JOIN classes
ON inscriptions.id_classe = classes.id_classe
WHERE classes.nom_classe = 'TG1';
```

Ici, le WHERE intervient après les jointures logiques :

- d'abord on relie les tables ;
- ensuite on sélectionne seulement les lignes qui concernent TG1.

Exemple dans un autre contexte : livres et auteurs

Table livres

id_livre	titre	id_auteur
1	Voyage au centre	3
2	Les Misérables	1
3	Notre-Dame	1

Table auteurs

id_auteur	nom
1	Hugo
2	Zola
3	Verne

Requête :

```
SELECT livres.titre, auteurs.nom
FROM livres
JOIN auteurs
ON livres.id_auteur = auteurs.id_auteur;
```

Résultat :

titre	nom
Voyage au centre	Verne
Les Misérables	Hugo
Notre-Dame	Hugo

Cet exemple montre bien la logique :

- la table livres contient l'identifiant de l'auteur ;
- le JOIN permet de remplacer cet identifiant par le nom réel.

Ce qu'il faut absolument retenir

Voici l'idée à faire graver chez les élèves :

1. Une base relationnelle sépare les informations en plusieurs tables

On évite les répétitions.

2. Les tables sont reliées par des identifiants

Souvent :

- clé primaire ;
- clé étrangère.

3. JOIN sert à reconstituer le lien entre les tables

On rassemble les informations utiles.

4. ON indique la règle de correspondance

Exemple :

ON table1.id = table2.id

5. WHERE ne sert pas à relier, mais à filtrer

Très important.

Formulation très simple à donner aux élèves

Tu peux leur dire ceci :

Un JOIN, c'est une façon de relier deux tables à partir d'une information commune, souvent un identifiant.

Et pour ON :

La partie ON indique quelle colonne de la première table correspond à quelle colonne de la deuxième.

Mini résumé ultra simple

```
SELECT colonnes
FROM table1
JOIN table2
ON table1.cle = table2.cle;
```

Traduction :

- je prends table1
- je la relie à table2
- quand les deux lignes ont la même clé
- puis j'affiche les colonnes demandées

Petit piège pédagogique à anticiper

Les élèves confondent souvent :

- la colonne qui sert à **identifier** ;
- et la colonne qui sert à **afficher**.

Exemple :

- on joint avec id_classe
- mais on affiche nom_classe

On ne fait pas la jointure sur le nom, sauf cas particuliers.
On la fait généralement sur l'identifiant.

Entraînement très court

Question 1

On a :

commandes

id_commande	id_client
1	5
2	8

clients

id_client	nom
5	Ali
8	Sarah

Écrire la requête pour afficher id_commande et nom.

Réponse :

```
SELECT commandes.id_commande, clients.nom
FROM commandes
JOIN clients
ON commandes.id_client = clients.id_client;
```

Question 2

Que fait ON commandes.id_client = clients.id_client ?

Réponse :
Il associe chaque commande au bon client en comparant les identifiants.

Une image mentale utile

On peut présenter JOIN comme une fermeture éclair :

- un côté = table A
- autre côté = table B
- les dents qui s'emboîtent = la condition ON

Si les bonnes dents correspondent, ça se ferme correctement.
Sinon, ça ne marche pas.

Conclusion

Pour bien comprendre JOIN, il faut toujours revenir à cette question :

"Quelle information commune permet de relier ces deux tables ?"

Si l'élève sait répondre à ça, il peut souvent écrire la condition de jointure.

Le plus dur n'est pas la syntaxe.
Le plus dur, c'est de bien comprendre **la structure des tables** et **le lien logique entre elles**.